متن ارائه‌ی آزمایش ۶ — طراحی انکوباتور

حدود ۵ دقیقه | پوریا عرفانزاده، فاطیما تیمارچی، آیه صابری

0:00 • شروع

سلام. من پوریا عرفانزاده هستم، با فاطیما تیمارچی و آیه صابری. آزمایش ششم ما طراحی واحد کنترل یک انکوباتور بود. تو این چند دقیقه می‌خوام بگم صورت مسئله چی بود، چه تصمیم‌های طراحی‌ای گرفتیم، چطور تستش کردیم، و نتیجه‌ی سنتز چی شد.

0:20 • صورت مسئله

یه محفظه داریم با یه سنسور دما، یه هیتر، و یه کولر که پنکه داره. سنسور دمای بین منفی ۱۰ تا مثبت ۶۰ درجه رو می‌خونه و به صورت یه عدد هشت‌بیتی علامت‌دار می‌ده به کنترلر. کنترلر باید دو تا تصمیم بگیره: اوّل اینکه هیتر و کولر کِی روشن و خاموش بشن، دوم اینکه اگه کولر روشنه، دور پنکه چقدر باشه. دستور کار این رفتار رو با دو تا state diagram داده، برای همین طرح ما هم طبیعتاً دو تا FSM شد، هر دو هم Moore.

0:50 • دو ماشین حالت

FSM اوّل سه تا حالت داره. S1 حالت idle که هر دو خاموشن. دما بره بالای ۳۵، می‌ره S2 که کولر روشنه و تا دما نیاد زیر ۲۵ برنمی‌گرده. از S1 دما بیاد زیر ۱۵، می‌ره S3 که هیتر روشنه و تا نره بالای ۳۰ برنمی‌گرده.

FSM دوم دور پنکه رو تنظیم می‌کنه: سه تا حالت کاری با دورهای ۴ و ۶ و ۸ دور بر ثانیه، به‌علاوه‌ی یه حالت OUT که یعنی غیرفعّال. بالای ۴۰ می‌ره روی ۶، بالای ۴۵ می‌ره روی ۸.

قید مهم، پیوند این دوتاست: FSM دوم فقط وقتی از OUT میاد بیرون که FSM اوّل تو

S2 باشه، یعنی کولر روشن باشه. منطقیه، وقتی کولر خاموشه تنظیم دورش معنی نداره. ما هم به جای اینکه دوباره عددهای ۳۵ و ۲۵ رو تکرار کنیم، مستقیم به حالت FSM اوّل نگاه کردیم؛ این‌جوری کلّی‌تره.

1:40 • تصمیم‌های طراحی

سه تا تصمیم طراحی رو می‌خوام برجسته کنم.

اوّل، همه‌ی نامساوی‌ها اکیدن. یعنی دمای دقیقاً ۳۵ گذار نمی‌سازه؛ از ۳۶ به بعد کولر روشن می‌شه. این رو جداگانه هم تست کردیم.

دوم، آستانه‌های رفت و برگشت با هم فرق دارن و همین hysteresis می‌سازه. کولر بالای ۳۵ روشن می‌شه ولی تا زیر ۲۵ خاموش نمی‌شه. این عمدیه و جلوی chattering رو می‌گیره، یعنی اینکه هیتر و کولر دمِ یه آستانه هی روشن و خاموش بشن. یه نتیجه‌ی جالبش اینه که اگه سیستم داره گرم می‌کنه، تو ۲۰ درجه هم هیتر هنوز روشنه.

سوم، سلامت سنسور. دستور کار نگفته بود اگه سنسور عدد بیرون از بازه بده چی کار کنیم، ولی چون ورودی علامت‌داره می‌تونه بده. ما یه سیگنال validity ساختیم و یه خروجی Error_Sensor و یه حالت fail-safe اضافه کردیم. نکته‌ی ظریفش اینه که validity مستقیم روی منطق خروجی اثر می‌ذاره، نه فقط روی گذار حالت؛ یعنی به‌محض دیدن مقدار نامعتبر هیتر و کولر بلافاصله خاموش می‌شن، بدون اینکه منتظر لبه‌ی کلاک بمونن. برای یه کنترلر ایمنی‌محور این رفتار درستیه.

یه نکته‌ی ریز هم بگم: برای فعّال کردن FSM پنکه از next_state ماشین اوّل استفاده کردیم نه current_state. اگه حالت جاری رو ملاک می‌ذاشتیم، دقیقاً تو همون لبه‌ای که کولر روشن می‌شه پنکه هنوز تو OUT بود و برای یه پالس کلاک، کولر روشن ولی دورش صفر گزارش می‌شد.

سؤال این بود که یه کنترلر رو اصلاً چطور باید تست کرد. اگه فقط دستی چند تا دما بدیم، داریم تو حلقه‌ی *باز* تست می‌کنیم؛ چون دمایی که می‌دیم ربطی به کاری که خود کنترلر داره می‌کنه نداره. تو واقعیّت ولی کنترلر هیتر رو روشن می‌کنه، دما بالا می‌ره، و همون دما دوباره ورودی خودشه. برای همین دو تا تست‌بنچ نوشتیم.

اوّلی حلقه‌بسته‌ست. داخل تست‌بنچ یه «انکوباتور قلّابی» نوشتیم — بهش می‌گن plant model — که فرمان‌های کنترلر رو می‌خونه و بر اساسش دمای محفظه رو هر «دقیقه» عوض می‌کنه، و خروجیش دوباره می‌ره روی ورودی کنترلر. یعنی حلقه بسته می‌شه و ما دیگه دما رو دیکته نمی‌کنیم؛ فقط شرط اوّلیّه و یه بار گرمایی بیرونی می‌ذاریم و تماشا می‌کنیم. سیستم از ۱۰ درجه‌ی سرد شروع می‌کنه، خودش گرم می‌کنه، بعد با بار بیرونی داغ می‌شه، کولر روشن می‌شه و دور پلّه‌پلّه می‌ره ۴ و ۶ و ۸، و آخر کار خودش برمی‌گرده به حالت خاموش. چند تا خطای سنسور هم بهش تزریق می‌کنیم که حالت امن هم تست بشه.

ولی این تست یه ضعف داره: مسیر دما طبیعیه، پس از هر آستانه فقط از یه سمت رد می‌شیم و هیچ‌وقت دقیقاً روی خودِ عدد آستانه نمی‌ایستیم. مثلاً وقتی دما ۴ درجه در دقیقه بالا می‌ره، از ۳۵ می‌پره به ۳۹.

پس تست‌بنچ دوم رو نوشتیم که برعکسه: مدلی در کار نیست، دما رو مستقیم و پلّه‌ای می‌دیم و بعد از هر پلّه هر چهار تا خروجی رو چک می‌کنیم. برای هر آستانه دو تا مقدار: خودِ آستانه که نباید گذار بده، و اوّلین مقدار بعدش که باید بده — مثلاً ۳۵ و بعد ۳۶. مرزهای اعتبار سنسور رو هم همین‌طور. در مجموع ۲۵ تا وارسی که همه‌شون pass شدن.

خلاصه اینکه اوّلی می‌گه سیستم در عمل درست کار می‌کنه، دوّمی می‌گه تک‌تک آستانه‌ها دقیقاً همونی‌ان که نمودار حالت خواسته.

4:00 • سنتز و رمزگذاری حالت

بعدش رفتیم Quartus. طرح خیلی کوچیکه، ۱۷ تا پین و زیر یک درصد ظرفیّت قطعه.

چون کلّ طرح از دو تا FSM ساخته شده، فرصت خوبی بود که اثر state encoding رو اندازه بگیریم. یه بار با binary کامپایل کردیم، یه بار با one-hot؛ کد کاملاً یکسان، فقط یه setting فرق داره.

نتیجه جالب بود: binary هم منابع کمتری گرفت، ۴۰ در برابر ۴۳ عنصر منطقی و ۵ در برابر ۸ رجیستر، و هم Fmax بالاتری داد، ۴۲۰ در برابر ۳۳۶ مگاهرتز. این خلاف انتظار متعارفه که می‌گه one-hot به قیمت مساحت سریع‌تره. دلیلش اینه که critical path این طرح اصلاً decode کردن حالت نیست، زنجیره‌ی مقایسه‌کننده‌های هشت‌بیتی دماست که قبل از منطق next-state قرار داره. با این تعداد کمِ حالت، decode دوبیتی تقریباً مجّانیه، ولی one-hot رجیستر بیشتری به مسیر اضافه می‌کنه. حرف کلّی‌ش اینه که rule of thumb جای اندازه‌گیری رو نمی‌گیره.

تحلیل زمانی رو هم با قید ۵۰ مگاهرتز و مدل بدترین شرایط زدیم؛ slack حدود ۱۷ نانوثانیه‌ست. البته تو کاربرد واقعی سنسور دقیقه‌ای یه بار نمونه می‌ده، پس اصلاً به این حدود نزدیک نمی‌شیم.

4:50 • جمع‌بندی

آخرش قطعه رو صریح انتخاب کردیم، هر ۱۷ تا پین رو با Pin Planner به پین فیزیکی نسبت دادیم با استاندارد ۲٫۵ ولت، دوباره کامپایل کردیم که منابع عوض نشد، و assembler بدون خطا فایل پیکربندی رو ساخت. چون آزمایشگاه مجازی بود روی برد فیزیکی پروگرام نکردیم و ادّعایی هم درباره‌ش نداریم.

جمع‌بندی: دو تا ماشین حالت هم‌کار با یه وابستگی شرطی، دو تا testbench که دو تا سؤال مختلف رو جواب می‌دن، و یه مقایسه‌ی encoding که نشون داد باید اندازه گرفت.

ممنون، اگه سؤالی هست در خدمتم.